

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – DEBUGGING**

Course Code:	CSA0901	Course Title:	Programming in Java ASSESSMENT TOOL 1 - DEBUGGING
CO Assessed:	CO3 – Precision (BL3,BL4)	Assessment:	
Weightage:		Total Marks:	100
CO1 Statement:	Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques		
Student Name:	S.Aarathana	Reg. No.:	1921260021
Section Batch: /	SLOT B	Date:	13/08/2026

Code of Conduct

I S.Aarathana (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

**Instructions**

- This test contains 4 questions Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Identifies all errors accurately	Identifies most errors	Identifies some errors	Unable to identify errors
Java Concept Understanding	25%	Demonstrates excellent understanding	Demonstrates good understanding	Demonstrates basic understanding	Demonstrates limited understanding
Debugging Skills	20%	Uses systematic debugging effectively	Uses appropriate debugging methods	Uses debugging with some guidance	Unable to debug independently
Code Correction	20%	Corrects all errors and produces correct output	Corrects most errors	Corrects some errors	Major errors remain
Testing & Explanation	15%	Tests thoroughly and clearly explains corrections	Tests and explains adequately	Limited testing/explanation	No proper testing or explanation

JAVA DEBUGGING

Questions & Fixed-Code Answers

Q1 • DEBUGGING QUESTION

Identify and correct the errors in the following program. Predict the output after debugging and explain the role of finally.

Debugging Task: *Correct the exception handling so that the program terminates normally after displaying an appropriate error message.*

FIXED CODE & OUTPUT

Your Fixed Code

```
1 class Test {
2     public static void main(String[] args) {
3         try {
4             int a = 10 / 0;
5             System.out.println("A = " + a);
6         }
7         catch (ArithmeticException e) {
8             System.out.println("Null Error");
9         }
10        finally {
11            System.out.println("Finally Block");
12        }
13        System.out.println("Program Completed");
14    }
15 }
16
17 /*Role of finally: It is mainly used for cleanup operations
18 such as closing files, database connections, or other resources.*/
```

Test Input

stdin input (optional)

Output

```
Null Error
Finally Block
Program Completed
```

Q2 • DEBUGGING QUESTION

Debug the following program so that it correctly throws and handles the user-defined exception when the balance is insufficient.

Debugging Task: *Identify the compilation error, modify the method appropriately, and ensure that the custom exception is correctly propagated and handled.*

FIXED CODE & OUTPUT

Your Fixed Code

```
1 public class InsufficientBalance extends Exception {
2
3     InsufficientBalance(String msg) {
4         super(msg);
5     }
6
7     static void withdraw(int balance, int amount)
8         throws InsufficientBalance {
9
10        if (amount > balance)
11            throw new InsufficientBalance("Insufficient Balance");
12
13        balance = balance - amount;
14        System.out.println("Remaining Balance: " + balance);
15    }
16
17    public static void main(String[] args) {
18        try {
19            withdraw(5000, 7000);
20        }
21        catch (InsufficientBalance e) {
22            System.out.println(e.getMessage());
23        }
24    }
25 }
```

Test Input

stdin input (optional)

Output

Insufficient Balance

Q3 • DEBUGGING QUESTION

The following program is intended to safely increment a shared counter using two threads. Debug it so that the final counter value is always 2000.

Debugging Task: Identify the synchronization and execution-order problems. Modify the program so that the final output is reliably: *Final Count = 2000*.

FIXED CODE & OUTPUT

Your Fixed Code

```
1 public class Counter {
2     int count = 0;
3
4     synchronized void increment() {
5         count++;
6     }
7
8     public static void main(String[] args) throws Exception {
9
10        Counter c = new Counter();
11
12        Thread t1 = new Thread(() -> {
13            for (int i = 0; i < 1000; i++)
14                c.increment();
15        });
16
17        Thread t2 = new Thread(() -> {
18            for (int i = 0; i < 1000; i++)
19                c.increment();
20        });
21
22        t1.start();
23        t2.start();
24
25        t1.join();
26        t2.join();
27
28        System.out.println("Final Count = " + c.count);
29    }
30 }
```

Test Input

stdin input (optional)

Output

Final Count = 2000

⌚ Pending manual review

Q4 • DEBUGGING QUESTION

The following program attempts to control a thread using deprecated thread-control methods.

Debugging Task: Identify all problems in the program, including the invalid thread priority and unsafe/deprecated thread-control methods. Rewrite it using safe thread-control mechanisms such as a flag and `interrupt()`, while maintaining the intended suspend/resume/stop behavior.

FIXED CODE & OUTPUT

Your Fixed Code

```
1 class Worker extends Thread {
2     private volatile boolean running = true;
3     private volatile boolean suspended = false;
4
5     public void run() {
6         for (int i = 1; i <= 10 && running; i++) {
7             while (suspended && running) {
8                 try {
9                     Thread.sleep(100);
10                } catch (InterruptedException e) {
11                    // Check flags again
12                }
13            }
14
15            if (!running)
16                break;
17
18            System.out.println("Working: " + i);
19
20            try {
21                Thread.sleep(200);
22            } catch (InterruptedException e) {
23                // Continue checking the flags
24            }
25        }
26    }
27
28    public void pauseWork() {
29        suspended = true;
30    }
31
32    public void resumeWork() {
33        suspended = false;
34        interrupt();
35    }
36
37    public void stopWork() {
38        running = false;
39        interrupt();
40    }
```

Test Input

stdin input (optional)

Output

```
Working: 1
Working: 2
Working: 3
Working: 4
Working: 5
Thread Suspended
```

```
42     public static void main(String[] args) throws Exception {
43         Worker w = new Worker();
44
45         w.setPriority(5);
46         w.start();
47
48         Thread.sleep(1000);
49
50         w.pauseWork();
51         System.out.println("Thread Suspended");
52
53         Thread.sleep(1000);
54
55         w.resumeWork();
56         System.out.println("Thread Resumed");
57
58         Thread.sleep(1000);
59
60         w.stopWork();
61         System.out.println("Thread Stopped");
62
63         w.join();
64     }
65 }
```